

ContextRotBench

Measuring Long-Horizon Degradation in Coding Agents

Dylan Lu* Surya Gunukula* Saaket Agashe Tengxiao Liu Xin Eric Wang
University of California, Santa Barbara * Equal contributions



Abstract

Long-running coding agents must continue applying earlier instructions as their interaction history grows. *Context rot* describes degradation as this context accumulates. Static retrieval tests leave reliability during ongoing agent interaction largely unmeasured.

We introduce **ContextRotBench**, a benchmark and reusable framework that keeps coding tasks fixed while varying task switches, interruptions, and filler text. We measure task performance and persistent instruction following to study how accumulated history affects agent behavior.

Benchmark design

We adapt software-engineering tasks into verifiable steps, then vary the history around those steps.

1 Task construction

Source SWE task

A sequence of target steps, each with a check.

Persistent instructions

Example: use `.format ( )` instead of f-strings throughout the task.

2 Context-rot run

Baseline

Interleaved

Interrupted

Filler

Side work runs in isolated containers. Only its conversation history carries forward.

3 Evaluation

Instruction following

% of active style checks satisfied.

Task performance

Source benchmark score after each target step.

Reported evaluation: 32 RoadmapBench tasks across Python, Go, and TypeScript, using OpenHands.

Results

Instruction following (%)

Model	Baseline	Interleaved	Interrupted	Filler
DeepSeek V4 Flash	77.4	73.6	72.1	74.9
Qwen3-Coder-30B	63.6	63.2	64.4	61.1
Nemotron 3 Ultra	75.6	71.1*	66.7*	-
Kimi K2.7 Code	87.3	79.6*	67.2	-
Kimi K2.6	78.4	61.9	45.3	75.2

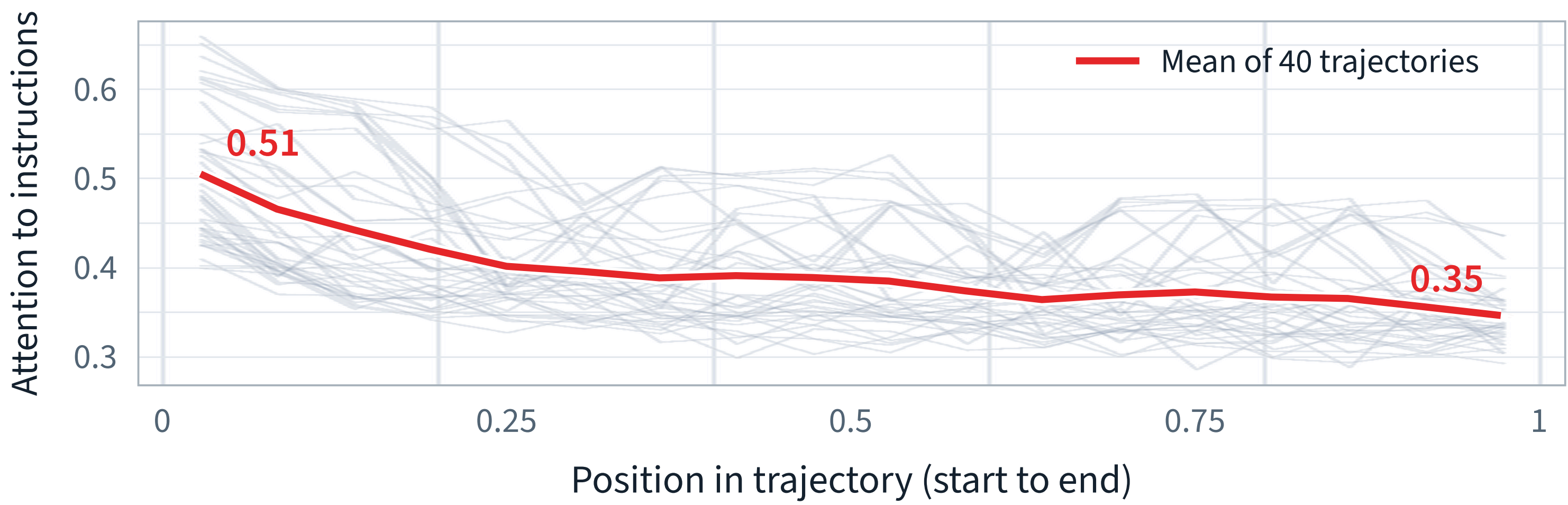
Decrease from each model's baseline: red >3 percentage points; bold red >6 percentage points.
* Incomplete evaluations with side-task refusal or no-action traces. - Missing run. Source: manuscript Table 2.

Task performance (%)

Model	Baseline	Interleaved	Interrupted	Filler
DeepSeek V4 Flash	28.7	23.6	23.9	25.5
Qwen3-Coder-30B	11.9	8.4	3.4	8.3
Nemotron 3 Ultra	12.9	5.0*	2.4*	-
Kimi K2.7 Code	16.9	17.1*	9.6*	-
Kimi K2.6	13.1	6.7*	15.3*	13.1

Intervening work can reduce instruction adherence. Task performance and instruction following do not always change together.

Attention analysis



Qwen3-Coder-30B, 40 trajectories, 48-layer mean. Attention to persistent instructions declines over the trajectory. Adapted from Fig. 3(b). Correlational evidence.

Discussion

ContextRotBench makes interaction history a controlled part of coding-agent evaluation. The results motivate evaluating whether agents continue applying earlier instructions while making progress on the task.

- For one target, a clean run satisfies 16 of 16 style checks, compared with 6 of 16 after four side-task turns.

Scope: 32 tasks, one agent framework, and a fixed set of interventions. Broader evaluation and methods for maintaining instruction adherence remain future work.